

บทที่ 5

การเลือก index อัตโนมัติ

เนื่องจากการเลือก Index terms เป็นงานที่ต้องใช้เวลา ความชำนาญ และความรู้เกี่ยวกับเนื้อหาภายในเอกสารประกอบกัน รวมทั้งอาจเกิดข้อผิดพลาดได้ง่าย จึงมีแนวความคิดที่จะใช้คอมพิวเตอร์มาทำหน้าที่แทนในส่วนนี้ โดยพิจารณาจากข้อมูลเชิงสถิติของการพบคำนั้ๆในเอกสาร และวิเคราะห์ด้วยวิธีการต่างๆ เพื่อให้ได้ข้อสรุปว่าคำนั้ๆสมควรถูกเลือกเป็น Index term หรือไม่

วิธีการวิเคราะห์เพื่อหา Index terms แบบอัตโนมัตินี้มีอยู่หลายวิธี แต่วิธีที่นิยมใช้ ได้แก่

5.1 การนับความถี่ของคำ (Frequency Count)

วิธีการนี้จะใช้ความถี่ของคำที่พบในเอกสารมาคำนวณตามที่กำหนด ซึ่งส่วนใหญ่แล้วจะเลือกคำที่มีความถี่มาก แต่ไม่มากเกินไป เป็น Index term ทั้งนี้เพราะคำที่พบมากๆในเอกสารนั้น ส่วนใหญ่มักเป็น Stopwords หรือคำที่ไม่มีความหมายต่อเอกสาร แต่คำสำคัญที่เกี่ยวข้องกับเอกสารนั้น มักพบมากกว่าคำปกติ แต่ไม่มากเกินไปนัก

นอกจากนี้เรายังสามารถเพิ่มความถูกต้องให้กับ การเลือก โดยใช้ตำแหน่งที่พบคำในเอกสารมาประกอบการคำนวณ โดยกำหนดค่าน้ำหนักต่อการพบคำในแต่ละส่วนให้ต่างกัน เช่น คำที่พบในส่วน Abstract ให้ค่าน้ำหนักมากกว่าคำที่พบในส่วนเนื้อเรื่อง เป็นต้น ซึ่งค่าน้ำหนักนี้จะถูกนำไปใช้เป็นข้อมูลในการเลือก Index terms ต่อไป

วิธีนี้เป็นวิธีพื้นฐานที่ทำได้ง่าย ไม่ซับซ้อน และมีประสิทธิภาพดีพอสมควร หากมีการกำจัด Stopwords ก่อนทำการเลือกได้อย่างมีประสิทธิภาพ แต่ข้อเสียของวิธีนี้ คือ กฎเกณฑ์ที่กำหนดไว้คงที่ ไม่มีการเปลี่ยนแปลง ทำให้ขาดความยืดหยุ่น

5.2 การใช้ Neural Network

วิธีการนี้ใช้แนวความคิดเหมือนกับวิธีการนับความถี่ของคำ ทั้งในส่วน of ข้อมูลที่จำเป็นต้องใช้ในการวิเคราะห์ และขั้นตอนการวิเคราะห์ แต่วิธีการคำนวณไม่คงที่ สามารถเปลี่ยนแปลงได้ โดยอาศัยการเรียนรู้ผ่าน Neural Network

วิธีนี้จะมีการทำงานที่ยู่ยากกว่าวิธีการนับความถี่คำ ใช้เวลาในการทำงาน และการเรียนรู้มาก มีการออกแบบและสร้างที่ยู่ยากกว่า แต่จะสามารถเรียนรู้จากข้อมูลใหม่ๆได้ ทำให้มีการปรับตัวให้ทำงานได้อย่างถูกต้องมากขึ้น

5.2.1 ทฤษฎี Neural Network

เป็นการจำลองการทำงานของสมองมนุษย์มาเป็นรูปแบบการประมวลผลข้อมูล ปกติแล้วในสมองของมนุษย์จะประกอบด้วยเซลล์ประสาทจำนวนมาก ซึ่งทำงานอย่างอิสระ แต่ละเซลล์จะมีการเชื่อมต่อระหว่างกันเรียกว่า Synapse มีส่วนที่เป็น Input ของเซลล์ เรียกว่า Dendrite และส่วนที่เป็น Output เรียกว่า Axon ซึ่งการติดต่อทำได้โดยการสร้างความต่างศักย์ (Voltage) ซึ่งถือว่าเซลล์ประสาทเหล่านี้เป็นหน่วยย่อยที่สุดในการทำงานของสมอง

ซึ่งการทำงานจะเป็นไปในลักษณะที่สัญญาณประสาทถูกส่งต่อกันไปเป็นทอดๆ จากเซลล์หนึ่งไปยังอีกเซลล์หนึ่ง โดยแต่ละเซลล์จะทำหน้าที่ในการคำนวณในส่วนย่อย และมีการเชื่อมต่อถึงกันที่ซับซ้อน เพื่อให้ได้ผลการคำนวณที่ต้องการ

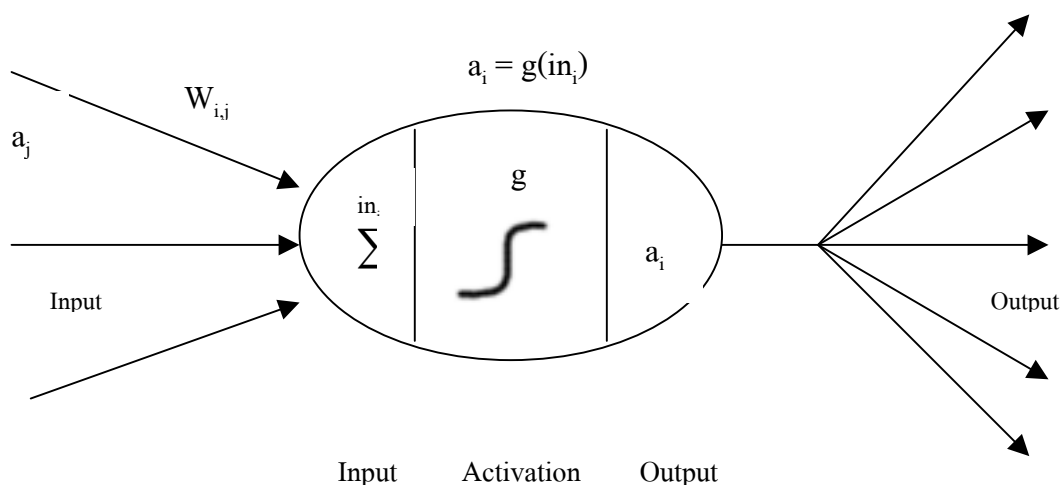
ข้อดีของการทำงานแบบนี้ คือ เราไม่จำเป็นต้องทราบถึงรูปแบบการคำนวณที่แน่นอน แต่ปล่อยให้ เป็นหน้าที่ของกระบวนการเรียนรู้ ที่จะทำหน้าที่สร้างรูปแบบการคำนวณที่ใกล้เคียงกับการคำนวณค่าที่ถูกต้อง ขึ้นมา โดยอาศัยการปรับค่าที่อยู่ภายในหน่วยย่อย หรือ ค่าที่บริเวณจุดเชื่อมต่อระหว่างหน่วยย่อยเหล่านี้ ว่าจะให้ค่านำเข้า (Input) ใดมีความสำคัญเท่าไร

เมื่อนำมาใช้ในทางคอมพิวเตอร์ เราจึงสร้างหน่วยประมวลผลซึ่งมีลักษณะคล้ายเซลล์ประสาท โดยให้แต่ละหน่วยประกอบด้วยส่วนรับข้อมูล (Input) ที่มีค่าน้ำหนัก (Weight) อยู่เพื่อระบุค่าความสำคัญของ Input นั้นๆ ภายในจะมีการประมวลผลโดยส่วนแรกจะนำค่า Input ทั้งหมดมารวมกัน จากนั้นจึงนำไปแทนค่าใน Activation Function แล้วได้ออกมาเป็น Output ดังแสดงไว้ในรูปที่ 5-1 ซึ่ง Output จากหน่วยหนึ่งจะมีค่าเพียงค่าเดียวเท่านั้น

เขียนเป็นสมการได้ ดังนี้

$$in_i = \sum_j W_{i,j} a_j$$

$$a_i = g(in_i)$$



รูปที่ 5-1 แสดงหน่วยประมวลผลใน Neural Network

เราสามารถเลือกใช้ Activation Function ได้หลายรูปแบบ แต่ที่นิยมใช้ที่สุดคือ Sigmoid Function เพราะสะดวกในการหา Derivative ซึ่งมักจำเป็นต้องใช้ในกระบวนการเรียนรู้ต่อไป

$$\text{sigmoid}(x) = 1 / (1 + e^{-x})$$

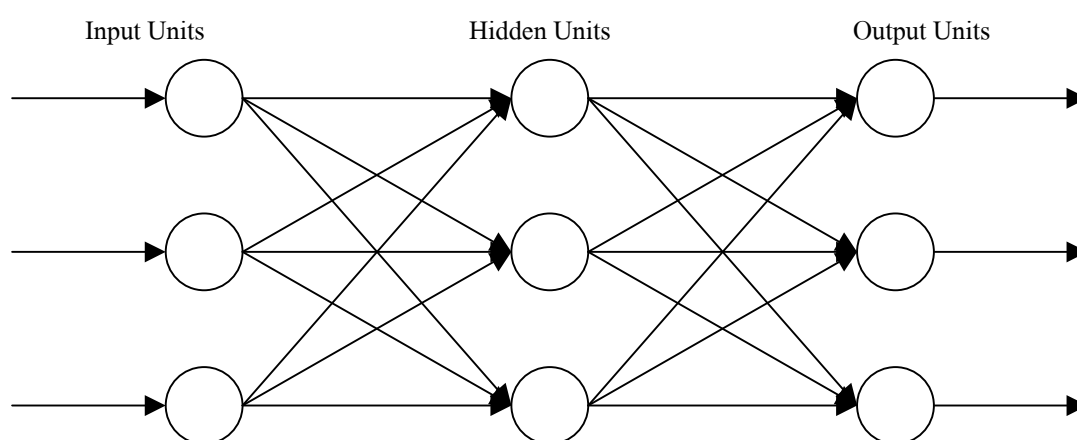
จากส่วนพื้นฐานนี้ สามารถนำมาต่อเป็น Network ได้หลายรูปแบบ ซึ่งแต่ละรูปแบบจะมีความเหมาะสมในการใช้งานแตกต่างกันไป

Algorithm	Type	Function
Hopfield	Recursive	Optimization
Multi-layered perceptron	Feedforward	Classification
Kohonen	Self-organizing	Data coding
Temporal difference	Predictive	Forecasting

ตาราง 5-1 แสดงรูปแบบการเชื่อมต่อพื้นฐานชนิดต่างๆ

โดยในแต่ละรูปแบบยังสามารถแบ่งเป็นรูปแบบย่อยๆ ที่แตกต่างกันไป หรือนำเอารูปแบบต่างๆ มาผสมกัน เพื่อให้ได้รูปแบบใหม่ ได้อีก ขึ้นกับการออกแบบ และความเหมาะสมในการนำไปใช้งานประเภทต่างๆ

ในที่นี้ เราจะให้ความสนใจกับ Multi-layered perceptron Network เท่านั้นดังแสดงรูปที่ 5-2 เนื่องจากเป็นชนิดที่นิยมใช้กันอย่างกว้างขวาง และเหมาะสมกับงานการจำแนกและแบ่งกลุ่มข้อมูล ตามรูปแบบ (Pattern) ซึ่ง Network แบบนี้จะมีลักษณะแบ่งออกเป็นชั้น (Layer) ซึ่งประกอบไปด้วย Input units, Output units และ Hidden units ซึ่งจำนวนชั้นของ Hidden units จะแตกต่างกันไปตามความซับซ้อนของการคำนวณที่ใช้ กล่าวคือ ยิ่งการคำนวณซับซ้อนมาก ก็ต้องมี Layers ของ Hidden Unit มากขึ้น



รูปที่ 5-2 ตัวอย่างของ Multi-layered Perceptron

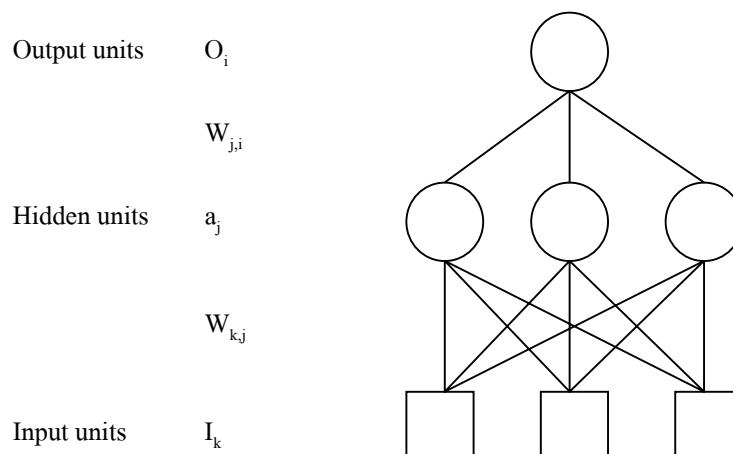
ข้อมูลที่ได้รับเข้ามาทาง Input Unit นั้นจะถูกผ่านไปให้แต่ละหน่วยประมวลผลที่อยู่ใน Layer เดียวกัน เพื่อทำการประมวลผล และสัญญาณเหล่านี้จะถูกส่งผ่านไปเรื่อยๆ ต่ Layer ถัดไป ซึ่งจะมีการให้ค่าความสำคัญ(Weight) ของ Input ที่รับเข้ามาจากหน่วยต่างๆแตกต่างกันไป และสัญญาณจะถูกส่งต่อระหว่าง Layer ไปเรื่อยๆ จนกระทั่งไปถึง Output unit ซึ่งจะส่งผลลัพธ์ที่ได้ออกมา

โดยค่าน้ำหนักของ Input ต่างๆนั้น จะถูกปรับโดยวิธีการเรียนรู้(Learning) โดยวิธีการเรียนรู้ ก็คือ การนำข้อมูลที่ทราบผลลัพธ์แล้ว มาใช้เป็น Input และเปรียบเทียบกับค่า Output ที่ได้กับผลลัพธ์หาค่าความผิดพลาด จากนั้นจึงนำค่าความผิดพลาดที่ได้มาทำการปรับน้ำหนัก (Weight) ภายใน Neural Network ซึ่งวิธีการปรับค่านี้นั้นขึ้นอยู่กับวิธีการเรียนรู้ที่เลือกใช้

ซึ่งวิธีการเรียนรู้ นั้น มีหลายรูปแบบ ขึ้นกับการออกแบบ โครงสร้างภายในของ Neural Network และประเภทของงานที่นำไปใช้

5.2.2 Back-propagation

เป็นวิธีการเรียนรู้รูปแบบหนึ่ง ซึ่งเป็นที่นิยมใช้ โดยอาศัยหลักการกระจายความรับผิดชอบต่อความผิดพลาดออกไปยังหน่วยย่อยทุกหน่วยใน Network เท่าๆกัน โดยจะกระจายความผิดพลาดลงไปในแต่ละ Layer ซึ่งจะสังเกตได้ว่าใน Multi-layered Perceptron Network นั้น ค่าที่ได้จากแต่ละ Layer จะส่งผลกระทบต่อ Layer ถัดไป ในลักษณะของการยกกำลัง ดังนั้นการกระจายความผิดพลาดจาก Layer หลังไปสู่ Layer ดันๆ นั้น จำเป็นต้องใช้หลักการอินทิเกรต เพื่อให้ได้อัตราความผิดพลาดที่เกิดขึ้นใน Layer นั้น



รูปที่ 5-3 แสดง Neural Network และค่าต่างๆ ที่ใช้ในการคำนวณ Back Propagation

จากรูปที่ 5-3 สามารถอธิบายได้ดังนี้ หากเรามี Network ที่ประกอบด้วย Input unit k, Hidden unit j และ Output unit i จะเขียนเป็นสมการได้ดังนี้

$$W_{j,i} \leftarrow W_{j,i} + (\alpha * a_j * \text{Err}_i * g'(in_i))$$

เมื่อ α แทน ค่าการเรียนรู้ และ Err_i แทน ผลต่างระหว่าง Output ของ i กับผลที่ถูกต้อง

$$\text{ให้ } \Delta_i = \text{Err}_i \cdot g'(\text{in}_i),$$

$$W_{j,i} \leftarrow W_{j,i} + (\alpha * a_j * \Delta_i)$$

ซึ่งเมื่อพิจารณาว่า Δ สำหรับ Layer ถัดไปจะได้ว่า

$$\Delta_j = g'(\text{in}_j) \sum_i W_{j,i} \Delta_i$$

$$W_{k,j} \leftarrow W_{k,j} + \alpha * I_k * \Delta_j$$

เมื่อ I_k แทน ค่าที่ได้จาก Input unit k

อัตราความเร็วของการเรียนรู้ขึ้นอยู่กับตัวแปร α ซึ่งหากมีค่ามากจะทำให้ค่าภายใน Neural Network เปลี่ยนแปลงไปอย่างรวดเร็วระหว่างการเรียนรู้ แต่ก็มีข้อเสียคือ จะทำให้ค่าที่ได้ไม่ละเอียด ซึ่งทำให้ผลการทำงานของ Neural Network ผิดพลาด ไม่เที่ยงตรงได้

ในกรณีที่เรายใช้ค่า α ไม่มากนัก อาจทำให้ในการทำ Back Propagation เพียงรอบเดียว ไม่สามารถปรับค่าให้ถูกต้องได้ ดังนั้นส่วนใหญ่ในการเรียนรู้แต่ละครั้งนั้น จะเป็นการทำ Back Propagation หลายๆรอบกับข้อมูลกลุ่มเดิม โดยยิ่งค่า α ต่ำ จำนวนรอบในการทำงานก็ยิ่งต้องเพิ่มมากขึ้น

นอกจากนี้ปริมาณและการกระจายตัวของข้อมูลตัวอย่างที่ใช้ในการเรียนรู้ (Training Set) นั้นก็เป็นสิ่งสำคัญที่มีผลต่อการเรียนรู้มาก ข้อมูลที่นำมาใช้เป็นตัวอย่างในการเรียนรู้นั้นต้องเป็นข้อมูลที่เป็นมาตรฐานตามปกติ ไม่ใช่ข้อมูลที่มีรูปแบบ โครงสร้าง ที่ผิดไปจากข้อมูลปกติ นอกจากนี้ต้องมีจำนวนมากพอที่จะทำให้มีรูปแบบที่หลากหลายพอที่จะครอบคลุมถึงลักษณะปกติของข้อมูลที่จะทำการวิเคราะห์ได้หมด